D3 Unit Test and Coverage Report

Story: Story B (Khushi): unit and system testing, defect history
Scope: B1: unit tests and coverage report
Repository: `CS455-Carpooling/Campus_Ridepooling_Automation`
Coverage run: 4 November 2026 (GitHub Actions run #242)
Tested commit: `3daef7f479da0b610b90d10dad6436ee5f72e96b`
Coverage provider: Vitest with V8

The figures in this report are from the completed CI run on the tested commit above. The workflow ran the unit suite with coverage, uploaded the HTML and JSON coverage output, and completed the production build. The PostgreSQL integration job also passed.

1. Execution summary

Metric	Result
Test files	201 passed / 201
Individual tests	1,535 passed / 1,535
Test duration	183.06 seconds
Statements	87.57%
Branches	80.73%
Functions	88.71%
Lines	89.92%

All 201 test files and all 1,535 tests passed. Each aggregate metric is above the 80% threshold configured in `apps/web/vitest.config.mts` for statements, branches, functions, and lines.

2. Coverage summary by area

Area percentages below are weighted from the per-file JSON coverage summary: covered counts are summed across the files in each area and divided by the corresponding total counts. The file counts include files with measurable statements in that area.

Area	Files	Statements	Branches	Functions	Lines
Library (`src/lib`)	73	88.29%	81.11%	93.99%	91.15%
API routes (`src/app/api`)	48	91.24%	84.49%	88.59%	94.60%
Components (`src/components`)	77	84.35%	78.53%	84.18%	85.87%

The area figures are diagnostic breakdowns. The configured 80% gate is applied to the aggregate project coverage; aggregate branch coverage is 80.73%. The component-area branch figure is 78.53%.

3. Files with the lowest statement coverage

The table lists the files with the lowest statement coverage in the CI JSON summary. Percentages are reported by Vitest; small files with only one or two statements are shown alongside larger files for completeness.

File	Statements	Branches	Functions	Lines
`src/instrumentation.ts`	0%	0%	0%	0%
`src/app/(app)/rides/page.tsx`	0%	0%	0%	0%
`src/app/(app)/rides/new/page.tsx`	0%	0%	0%	0%
`src/app/(app)/rides/[id]/complaint/loading.tsx`	0%	100%	0%	0%
`src/app/(app)/rides/results/loading.tsx`	0%	100%	0%	0%
`src/app/admin/configuration/loading.tsx`	0%	100%	0%	0%
`src/app/profile/page.tsx`	0%	0%	0%	0%
`src/lib/ai/gemini-ranking.ts`	0%	0%	0%	0%
`src/components/rides/JoinRequestControls.tsx`	23.40%	15.23%	12.50%	23.91%
`src/components/home/DashboardTabs.tsx`	47.36%	50%	80%	47.36%

A zero-coverage result is a measurement from this test run; by itself it does not establish that the file contains a defect. Loading and framework entry files may have very little measurable code.

4. Tests included in this update

Five tests in `apps/web/src/lib/ai/shared-interest-enrichment.test.ts` exercise `enrichSharedInterests` with the database pool's `pool.query` mocked:

1. An empty candidate list returns an empty map without querying the database pool.
2. Duplicate candidate ride IDs are deduplicated for the query and result map; candidates without returned rows remain unavailable (`null`).
3. Approved shared labels are returned in normalized, deduplicated order; candidates without returned rows remain unavailable.
4. Shared labels are withheld when participant consent is incomplete.
5. No common interest is represented as an empty list, while missing visible-tag data is represented as unavailable (`null`).

The four direct `resolveSharedInterestTags` cases are retained in `apps/web/src/lib/ai/shared-interest-tags.test.ts`:

- approved labels are sorted and deduplicated when all participants consent and have visible tags;
- labels are withheld when any participant has not consented;
- labels are withheld when any participant lacks usable visible tags;
- no common interest with otherwise usable data returns an empty list.

Together, these changes preserve direct helper coverage while testing the enrichment flow that invokes the helper.

5. Reproduction instructions

From the repository root, install dependencies and run the web workspace's test script:

```
npm ci
cd apps/web
npm test
```

The tested commit was merged into `master` as `e494209` with no further code change, so the same run can be repeated on master:

```
git switch master
git pull origin master
npm ci
cd apps/web
npm test
```

A local rerun on master on 4 November 2026 gave the same results: 201 files, 1,535 tests, and the coverage figures in Section 1.

The workspace's `test` script is `vitest run --coverage`; do not append another `--coverage` argument.

6. CI evidence

- Workflow: CI run #242
- Tested commit: `3daef7f479da0b610b90d10dad6436ee5f72e96b`
- Unit job: format, lint, type-check, tests, coverage artifact upload, and production build all passed.
- Integration job: PostgreSQL integration tests passed.
- Coverage artifact: Download unit coverage HTML and JSON output

The CI coverage artifact contains the HTML report and machine-readable `coverage-summary.json` used for the aggregate and area calculations above. GitHub keeps the artifact for 14 days only, so the same files are also included in this deliverable as `unit-coverage-report.zip` (open `coverage/index.html`).

7. Final run on master

Tests were added after the tested commit above, so the suite was run again on the final code on 6 November 2026. All 202 test files and all 1,544 tests passed, with 87.67% statements, 80.91% branches, 88.79% functions and 89.98% lines, each above the 80% threshold. The Code Coverage Report gives the breakdown of this run, and the traceability matrix uses it.